

IBM - BUILDING RAG APPLICATIONS



Summarize Private Documents Using RAG, LangChain, and LLMs.

RAG architecture

A typical RAG application has two main components:

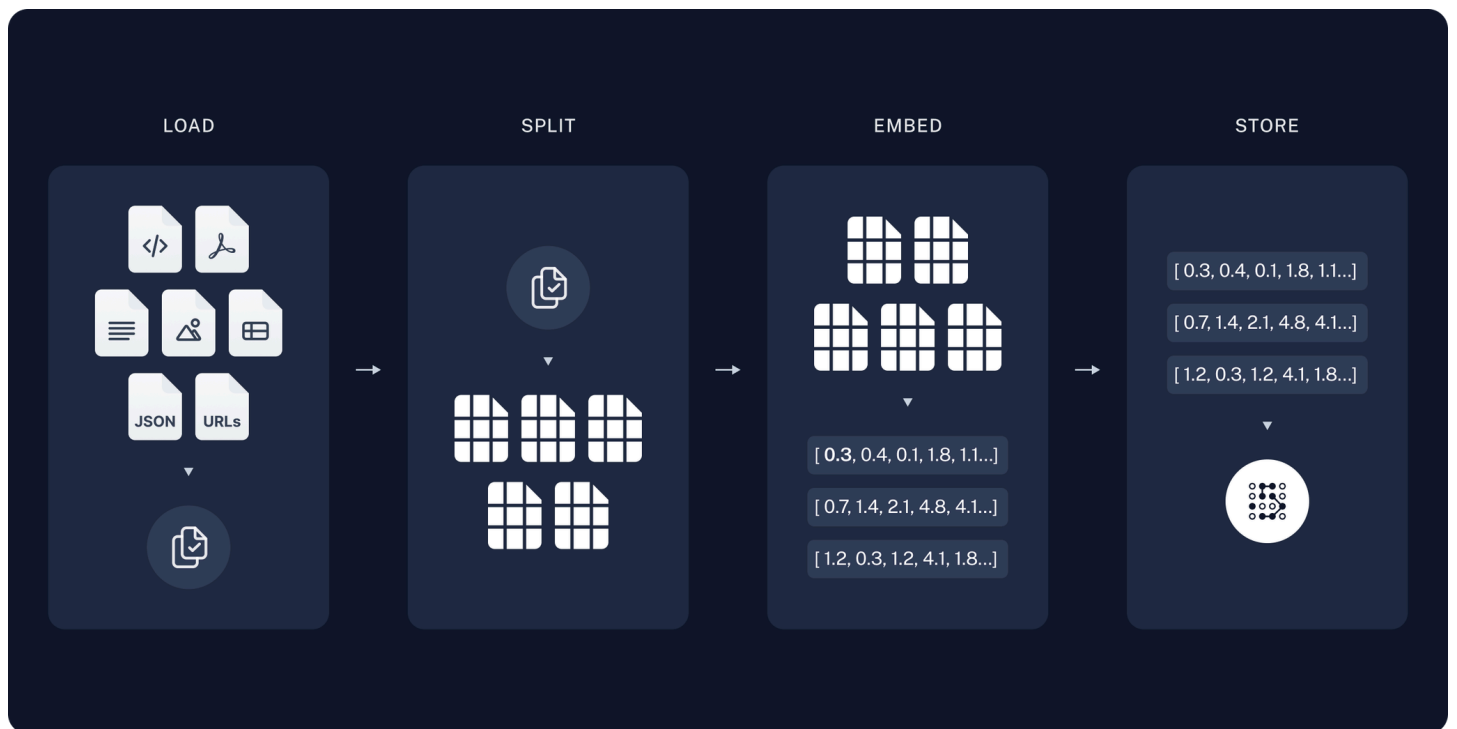
- **Indexing:** A pipeline for ingesting and indexing data from a source. This usually happens offline.
- **Retrieval and generation:** The actual RAG chain takes the user query at run time and retrieves the relevant data from the index, then passes that to the model.

Indexing

1. Load: First, you must load your data. This is done with [DocumentLoaders](#).
2. Split: [Text splitters](#) break large Documents into smaller chunks. This is useful both for indexing data and for passing it into a model because large chunks are harder to search

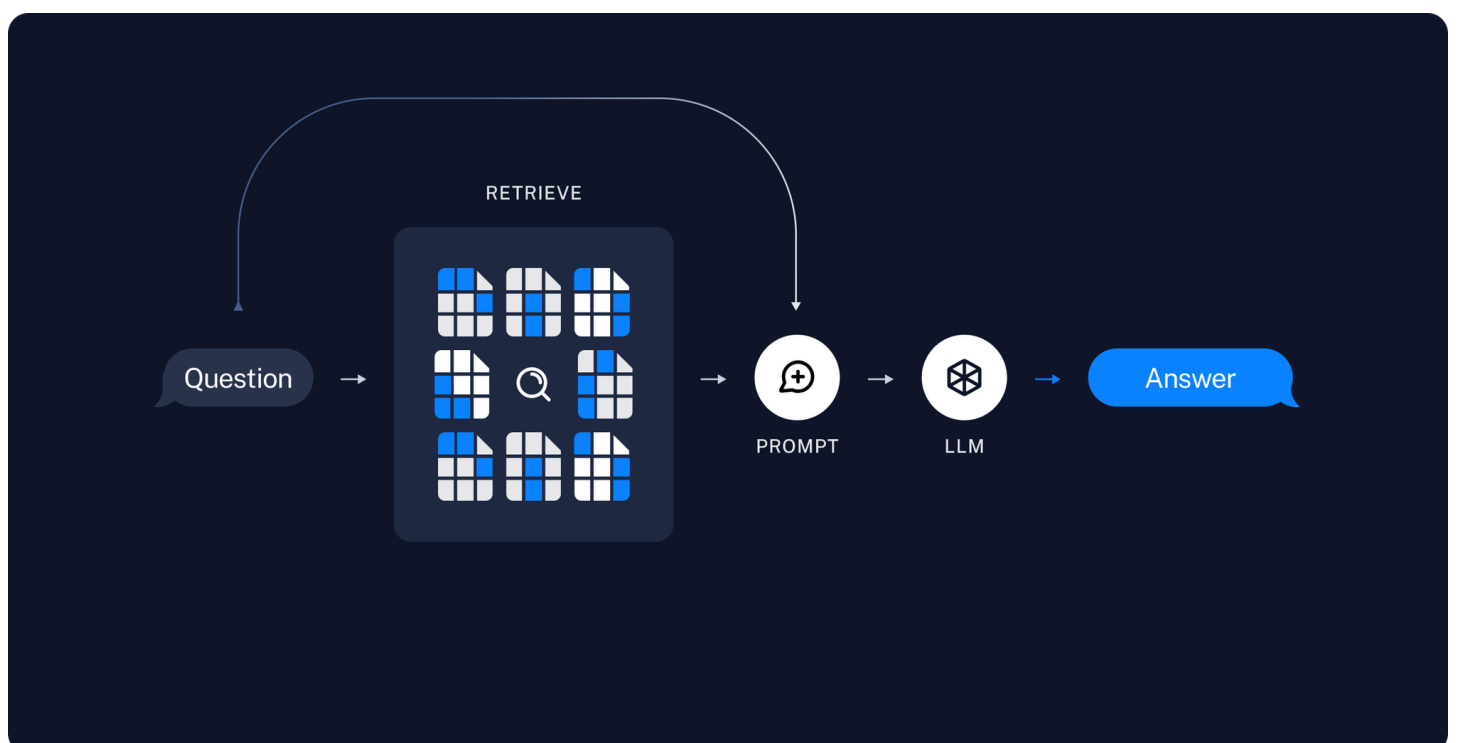
and won't fit in a model's finite context window.

3. Store: You need somewhere to store and index your splits so that they can later be searched. This is often done using a [VectorStore](#) and [Embeddings](#) model.



Retrieval and generation

1. Retrieve: Given a user input, relevant splits are retrieved from storage using a retriever.
2. Generate: A ChatModel / LLM produces an answer using a prompt that includes the question and the retrieved data.



SETUP

For this lab, you are going to use the following libraries:

- [ibm-watsonx-ai](#) for using LLMs from IBM's watsonx.ai
- [LangChain](#) for using its different chain and prompt functions
- [Hugging Face](#) and [Hugging Face Hub](#) for their embedding methods for processing text data
- [SentenceTransformers](#) for transforming sentences into high-dimensional vectors
- [Chroma DB](#) for efficient storage and retrieval of high-dimensional text vector data
- [wget](#) for downloading files from remote systems

Library	Primary Role	Function in RAG Pipeline
ibm-watsonx-ai	LLM Provider	Connects to enterprise foundation models hosted on IBM Cloud to generate final answers from augmented prompts.
LangChain	Framework Orchestrator	Coordinates the overall workflow, managing prompt templates, retriever chains (RetrievalQA), and data movement.
Hugging Face / Hugging Face Hub	Model Ecosystem	Provides access to open-source model repositories and embedding architectures for downloading text processing tools.
SentenceTransformers	Embedding Engine	Converts text chunks and user queries into dense, high-dimensional vector representations that capture semantic meaning.
Chroma DB	Vector Database	Stores high-dimensional vector embeddings with metadata and performs fast similarity searches (e.g., Cosine/Dot Product).
wget	Data Retrieval Utility	Downloads raw files, datasets, or reference documents from remote servers into the workspace for processing.

INSTALLING LIBRARIES

```
[1]: %%capture
      %pip install -U \
      langchain \
      langchain-core \
      langchain-community \
      langchain-text-splitters \
      langchain-huggingface \
      langchain-chroma \
      langchain-classic \
      langchain-ibm \
      ibm-watsonx-ai \
      chromadb \
      sentence-transformers \
      transformers \
      huggingface-hub \
      wget
```

```
[1]: %pip install --upgrade \
      numpy \
      pandas \
      scipy \
      scikit-learn
```

Hugging Face / Hugging Face Hub	Model Ecosystem	Provides access to open-source model repositories and embedding architectures for downloading text processing tools. TXT
SentenceTransformers	Embedding Engine	Converts text chunks and user queries into dense, high-dimensional vector representations that capture semantic meaning. TXT

Importing required libraries

```

def warn(*args, **kwargs):
    pass

import warnings
warnings.warn = warn
warnings.filterwarnings("ignore")

import wget

# LangChain
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import CharacterTextSplitter
from langchain_chroma import Chroma
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_classic.chains import RetrievalQA, ConversationalRetrievalChain
from langchain_classic.memory import ConversationBufferMemory
from langchain_core.prompts import PromptTemplate

# IBM watsonx
from ibm_watsonx_ai.foundation_models import Model
from ibm_watsonx_ai.metanames import GenTextParamsMetaNames as GenParams
from ibm_watsonx_ai.foundation_models.utils.enums import (
    ModelTypes,
    DecodingMethods,
)
from langchain_ibm import WatsonxLLM

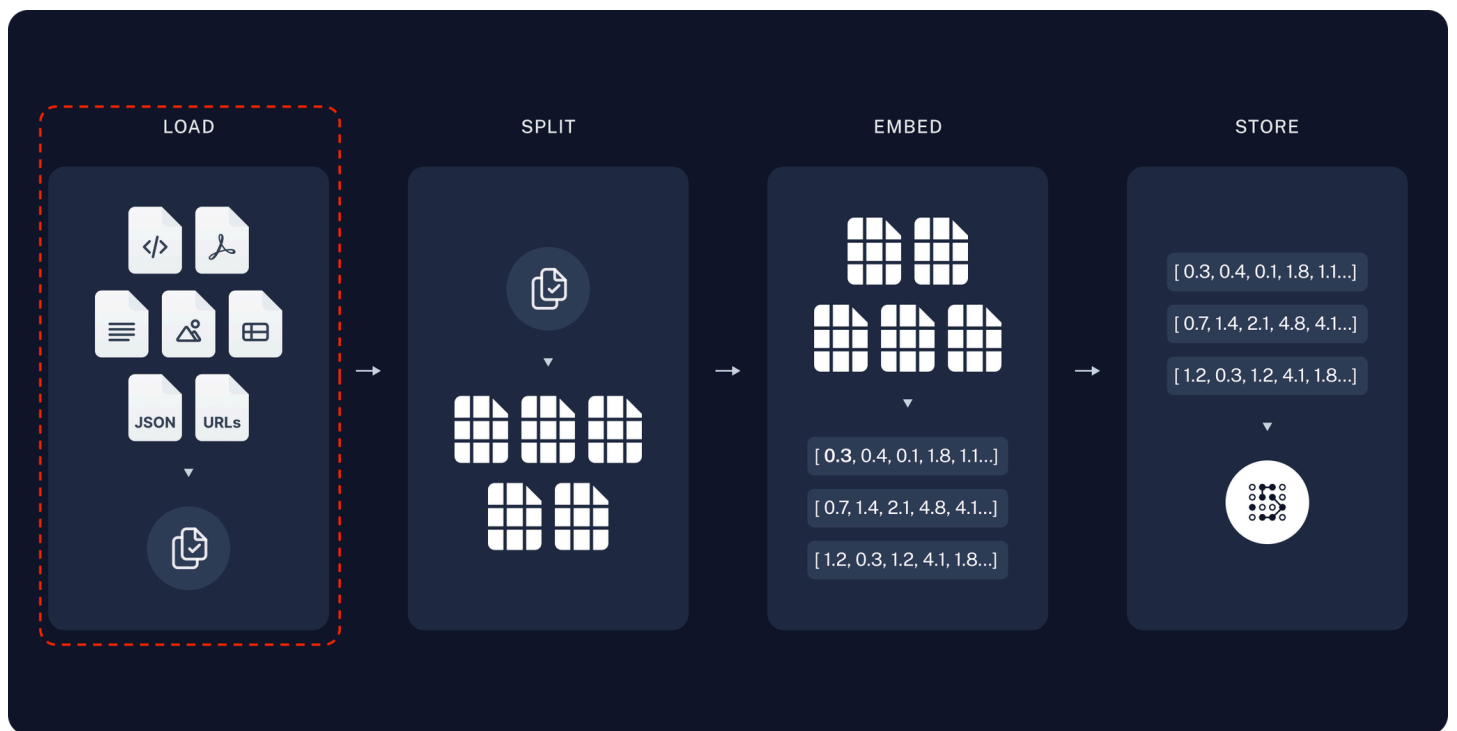
print("All imports successful!")

```

Preprocessing

Load the Document

This is the load step in Indexing.



```
filename = 'companyPolicies.txt'
url = 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/courses-data/Chatbot/companyPolicies.txt'

# Use wget to download the file
wget.download(url, out=filename)
print('file downloaded')
```

file downloaded

```
with open(filename, 'r') as file:
    contents = file.read()
    print(contents)
```

Acceptable Use: Mobile devices are primarily intended for work-related use.
Security: Safeguard your mobile device and access credentials. Exercise caution when connecting to public Wi-Fi networks.
Confidentiality: Avoid transmitting sensitive company information through mobile devices.
Cost Management: Keep personal phone usage separate from company-provided devices.
Compliance: Adhere to all pertinent laws and regulations concerning mobile device use.
Lost or Stolen Devices: Immediately report any lost or stolen mobile devices to the IT department.
Consequences: Non-compliance with this policy may lead to disciplinary action.
The Mobile Phone Policy is aimed at promoting the responsible and secure use of mobile devices.

5. Smoking Policy

Policy Purpose: The Smoking Policy has been established to provide a safe and healthy work environment for all employees. Designated Smoking Areas: Smoking is only permitted in designated areas.

Splitting the Documents into Chunks

In this step, you are splitting the document into chunks, which is basically the split process in Indexing.

For the splitting process, the goal is to ensure that each segment is as extensive as if you were to count to a certain number of characters and meet the split separator. This certain number is called chunk size.

Let's set 1000 as the chunk size in this project. Though the chunk size is 1000, the splitting is happening randomly.

This is an issue with LangChain. CharacterTextSplitter uses `\n\n` as the default split separator.

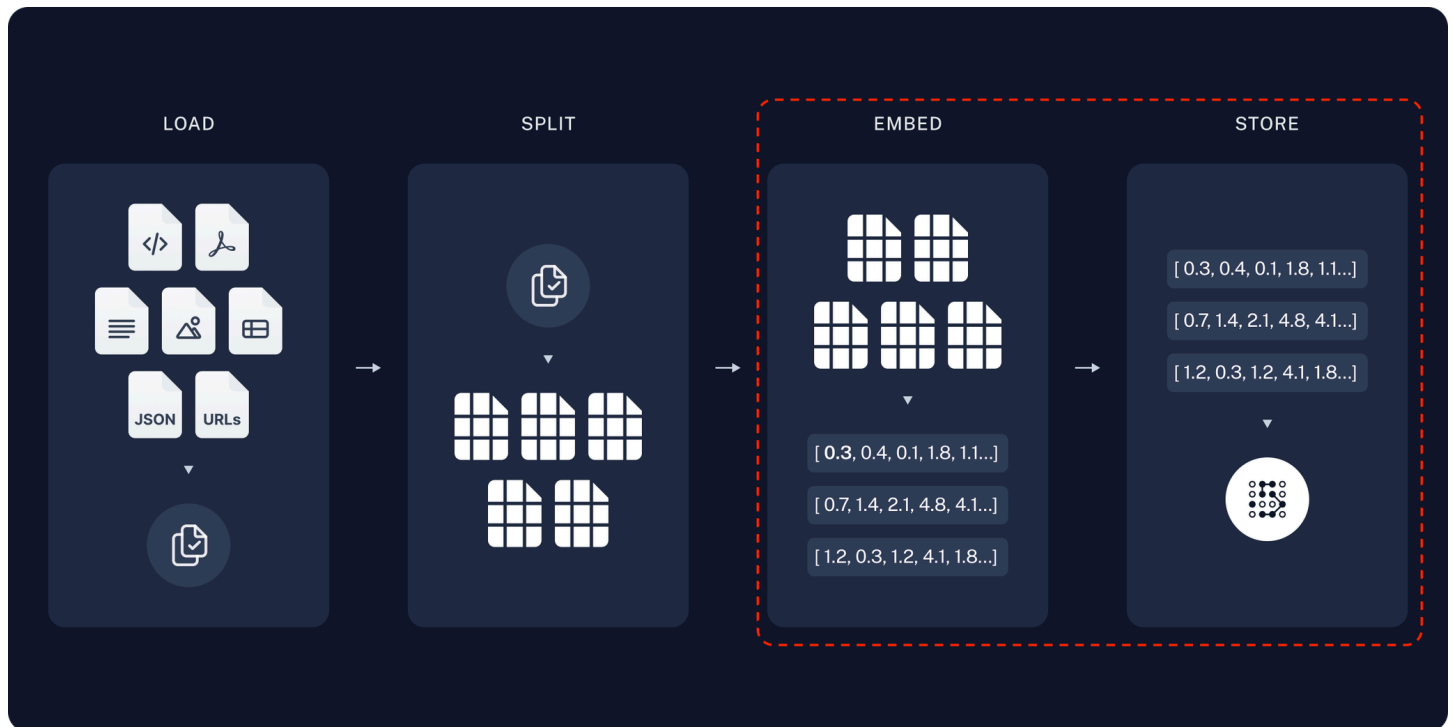
You can change it by adding the separator parameter in the CharacterTextSplitter function; for example, `separator="\n"`.

```
loader = TextLoader(filename)
documents = loader.load()
text_splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=0)
texts = text_splitter.split_documents(documents)
print(len(texts))

... WARNING:langchain_text_splitters.base:Created a chunk of size 1624, which is longer than the specified 1000
WARNING:langchain_text_splitters.base:Created a chunk of size 1885, which is longer than the specified 1000
WARNING:langchain_text_splitters.base:Created a chunk of size 1903, which is longer than the specified 1000
WARNING:langchain_text_splitters.base:Created a chunk of size 1729, which is longer than the specified 1000
WARNING:langchain_text_splitters.base:Created a chunk of size 1678, which is longer than the specified 1000
WARNING:langchain_text_splitters.base:Created a chunk of size 2032, which is longer than the specified 1000
WARNING:langchain_text_splitters.base:Created a chunk of size 1894, which is longer than the specified 1000
16
```


Embedding and storing

This step is the embed and store processes in Indexing.



converting the text into numbers, and making them easier for your computer to understand and remember by using a process called "embedding".

```
embeddings = HuggingFaceEmbeddings()
docsearch = Chroma.from_documents(texts, embeddings)
print('document ingested')
```

LLM MODEL CONSTRUCTION

```
model_id = 'meta-llama/llama-4-maverick-17b-128e-instruct-fp8'

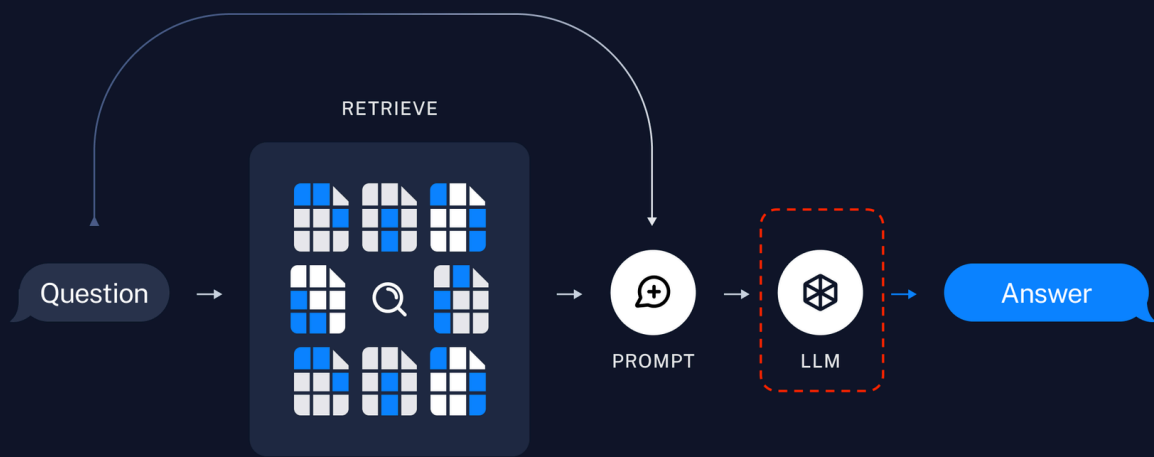
parameters = {
    GenParams.DECODING_METHOD: DecodingMethods.GREEDY,
    GenParams.MAX_NEW_TOKENS: 256,
    GenParams.MIN_NEW_TOKENS: 130,
    GenParams.TEMPERATURE: 0.5
}

credentials = {
    "url": "https://us-south.ml.cloud.ibm.com"
}

project_id = "skills-network"

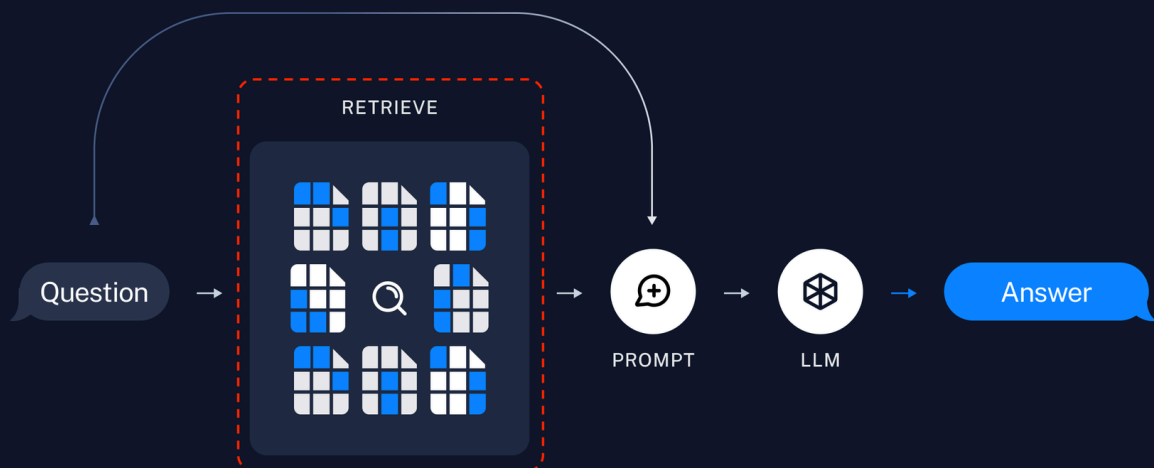
llm = WatsonxLLM(
    model_id=model_id,
    url="https://us-south.ml.cloud.ibm.com",
    project_id=project_id,
    params=parameters,
)
```

This completes the LLM part of the Retrieval task.



INTEGRATING LANGCHAIN

LangChain has a number of components that are designed to help retrieve information from the document and build question-answering applications, which helps you complete the retrieve part of the Retrieval task.



```
qa = RetrievalQA.from_chain_type(llm=llm,  
                                chain_type="stuff",  
                                retriever=docsearch.as_retriever(),  
                                return_source_documents=False)  
  
query = "what is mobile policy?"  
qa.invoke(query)
```